

Relazione PSS25-Chronio
“Progettazione e Sviluppo Software” A.A.
2025/2026

Aurora Loprino - aurora.loprino@studio.unibo.it
Enrico Poli - enrico.poli@studio.unibo.it

28 giugno 2026

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	3
2	Design	5
2.1	Architettura	5
2.2	Design dettagliato	6
2.2.1	Aurora Loprino	6
2.2.2	Enrico Poli	11
3	Sviluppo	17
3.1	Testing automatizzato	17
3.2	Note di sviluppo	18
3.2.1	Aurora Loprino	18
3.2.2	Enrico Poli	19
4	Commenti finali	21
4.1	Autovalutazione e lavori futuri	21
A	Guida utente	23

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Chronio è un'applicazione per la gestione personale del tempo e delle attività. L'utente può tenere traccia dei propri impegni tramite un calendario, organizzare attività in bacheche kanban, e monitorare le proprie finanze personali tramite un modulo di budget.

Requisiti funzionali

- L'applicazione deve consentire la creazione, modifica ed eliminazione di eventi nel calendario, con supporto a eventi giornalieri e a tutto il giorno.
- Gli eventi devono poter essere associati a tag colorati, creabili, modificabili ed eliminabili dall'utente.
- I tag devono poter essere resi invisibili per filtrare gli eventi nella vista.
- Il calendario deve offrire una vista mensile, settimanale e giornaliera, con navigazione tra periodi.
- L'applicazione deve mostrare una barra laterale con gli eventi del giorno corrente e dei successivi sei giorni.
- L'applicazione deve consentire la creazione di bacheche kanban, ciascuna con colonne e card personalizzabili.
- Le card devono poter essere associate a tag, marcate come completate e filtrate per tag.

- L'applicazione deve consentire la gestione di un budget personale con transazioni e tag.
- I dati devono essere persistiti su disco e ricaricati automaticamente all'avvio.

Requisiti non funzionali

- L'applicazione deve essere eseguibile su Windows, macOS e Linux senza configurazione aggiuntiva da parte dell'utente.
- I dati devono essere salvati automaticamente dopo ogni operazione, senza intervento esplicito dell'utente.
- L'interfaccia deve essere reattiva e non bloccarsi durante le operazioni di salvataggio.

1.2 Modello del Dominio

Il dominio di Chronio è composto da tre aree principali: calendario, bacheche kanban e budget.

Nel calendario, l'entità centrale è l'**Evento**, caratterizzato da titolo, descrizione, data + ora di inizio e fine opzionale, un flag per gli eventi a tutto il giorno, e un riferimento opzionale a un **Tag**.

Il Tag è un'etichetta colorata con un nome e uno stato di visibilità.

Il **Calendario** raccoglie tutti gli eventi e i tag, e consente di interrogarli per data.

Nelle bacheche, la **Bacheca** è l'entità principale, composta da **Colonne** ordinate. Ogni colonna contiene un insieme ordinato di **Card**, ciascuna con titolo, descrizione, stato di completamento e una lista di **KanbanTag** associati.

Gli elementi del dominio e le relazioni fra loro sono rappresentati in Figure 1.1.

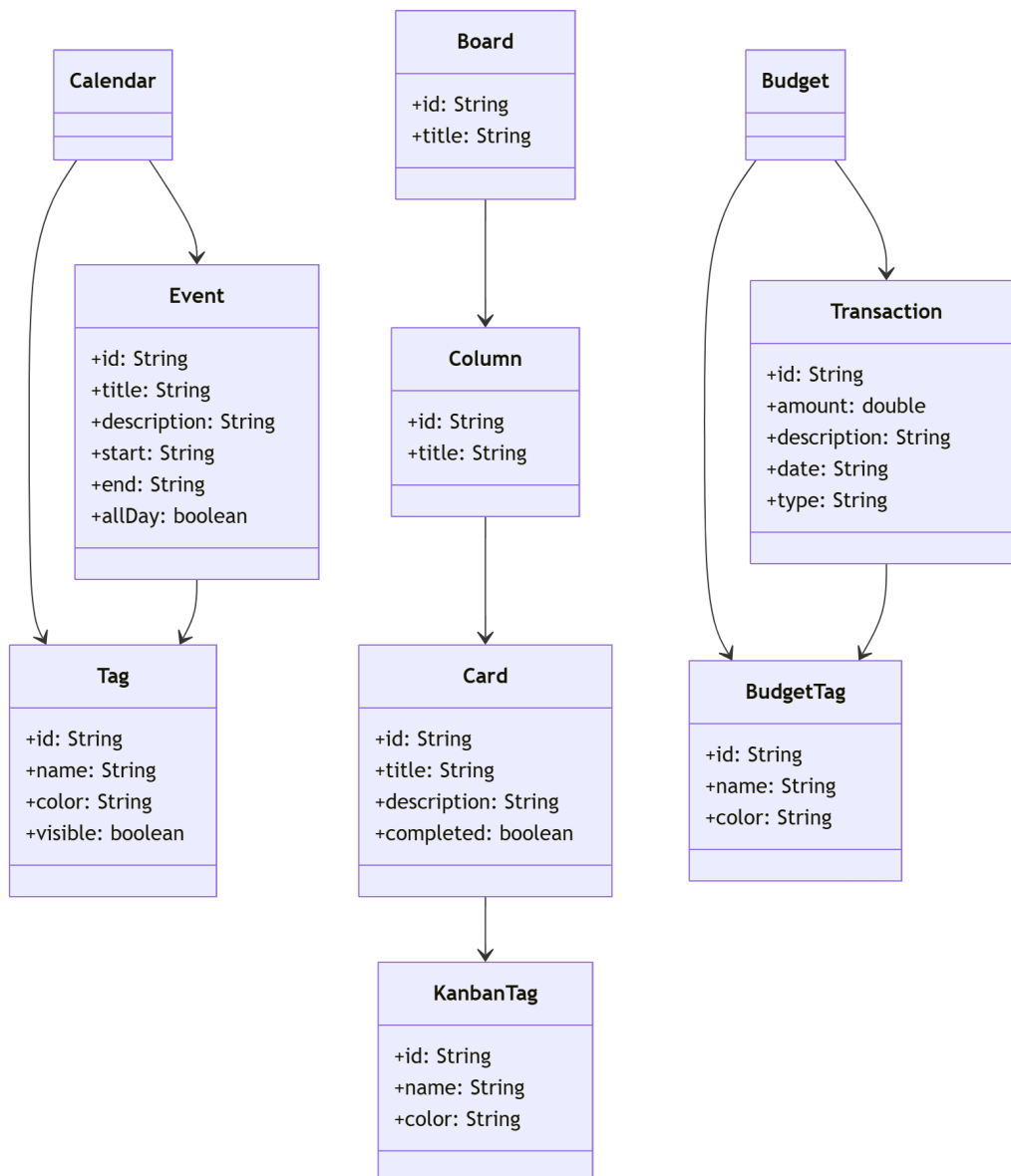


Figura 1.1: Schema UML del dominio di Chronio con entità principali e relazioni.

Capitolo 2

Design

2.1 Architettura

L'architettura di Chronio segue il pattern **MVC (Model-View-Controller)** applicato a ciascuna delle tre aree funzionali (calendario, kanban, budget).

Ogni area ha il proprio model, controller e view, tra loro indipendenti.

Il **Model** contiene le entità del dominio (record Java immutabili) e la logica di business quale la validazione delle date, il filtraggio per tag e il calcolo degli eventi per data.

Il **Controller** fa da intermediario tra view e model, delegando la logica al model e invocando la persistenza dopo ogni operazione di scrittura.

La **View** è costruita interamente in JavaFX, limitandosi a chiamare il controller e a ridisegnarsi in risposta agli eventi utente.

La persistenza è separata dal model: ogni area ha la propria classe di persistenza che salva e carica i dati in formato JSON tramite Gson.

Sostituire la view con un'altra tecnologia grafica non richiederebbe modifiche al model né al controller, dato che nessuna delle interfacce di model e controller dipende da JavaFX.

Il modulo kanban non ha un'interfaccia **BoardModel** separata: tutta la logica risiede in **BoardControllerImpl**, che gestisce direttamente il record **BoardData**. La scelta di non introdurre un'interfaccia **BoardModel** separata è motivata dall'assenza di logica di dominio complessa nel modulo. Le operazioni sono per lo più di CRUD su strutture dati ordinate, già testabili direttamente tramite **BoardControllerImpl**.

Lo schema architetturale è sintetizzato in Figure 2.1.

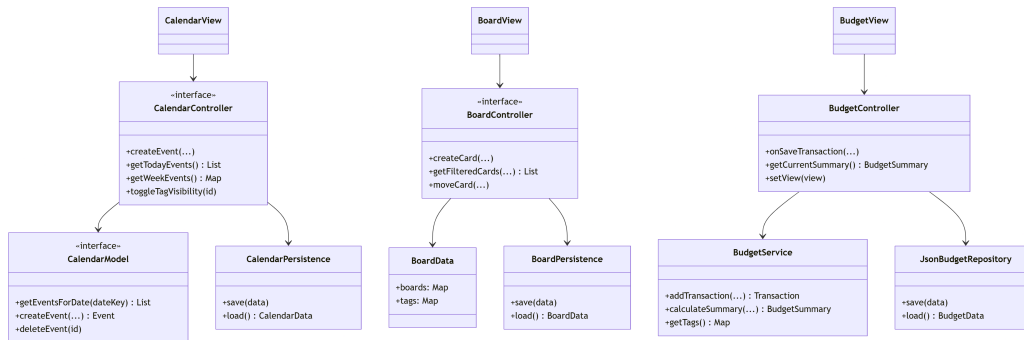


Figura 2.1: Schema UML architetturale di Chronio. Ogni modulo applica il pattern MVC in modo indipendente.

2.2 Design dettagliato

2.2.1 Aurora Loprino

Immutabilità del modello con record Java

Problema Le entità del dominio devono essere condivise tra model, controller e view senza rischio di modifiche accidentali. In un sistema con salvataggio automatico dopo ogni operazione, è essenziale che lo stato non venga mutato in modo implicito.

Soluzione Il modello del calendario e delle bacheche è rappresentato interamente tramite record Java (`Event`, `Tag`, `Board`, `Column`, `Card`, `KanbanTag`). I record sono immutabili, quindi ogni operazione di modifica produce un nuovo oggetto; questo elimina la possibilità di modifiche accidentali dello stato e rende il modello thread-safe per lettura.

L'aggiornamento dello stato avviene sostituendo l'intera struttura dati (`CalendarData`, `BoardData`) con una nuova istanza contenente i dati aggiornati.

La struttura del model del calendario è mostrata in Figure 2.2.

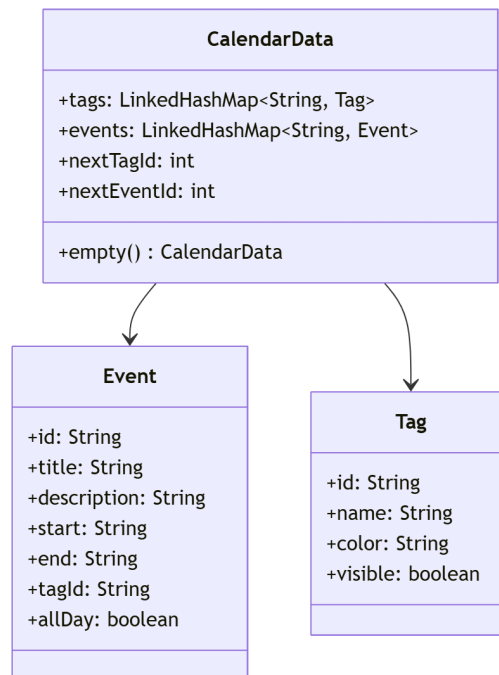


Figura 2.2: **CalendarData** come record immutabile che aggrega eventi e tag.

Filtraggio degli eventi per tag visibili

Problema Gli eventi associati a un tag nascosto non devono comparire in nessuna vista (mensile, settimanale, giornaliera, sidebar). La logica di filtraggio deve essere centralizzata per evitare duplicazioni tra le diverse viste.

Soluzione Il model gestisce la visibilità degli eventi in base allo stato del tag associato. Il metodo **getEventsForDate** itera su tutti gli eventi in **CalendarData** applicando due metodi privati. **isVisible** restituisce **true** se l'evento non ha tag o se il tag associato è visibile, e **fallsOn** verifica se la data richiesta ricade nell'intervallo start–end dell'evento, supportando gli eventi multi-giorno. Questo consente alla view di non contenere alcuna logica di filtraggio.

Vista del calendario a tre livelli

Problema Il calendario deve offrire tre tipi di visualizzazione (mese, settimana, giorno) navigabili senza ricreare la struttura principale della finestra.

Soluzione La vista è organizzata su tre livelli (`CalendarView`, `WeekView`, `DayView`), navigabili tramite un unico pulsante di toggle. La navigazione avviene sostituendo il contenuto della `VBox` principale senza ricreare il nodo radice, preservando la navbar. Le tre viste condividono i metodi di utilità tramite la classe `ViewUtils`, eliminando la necessità di duplicare il codice.

La gerarchia delle viste è illustrata in Figure 2.3.

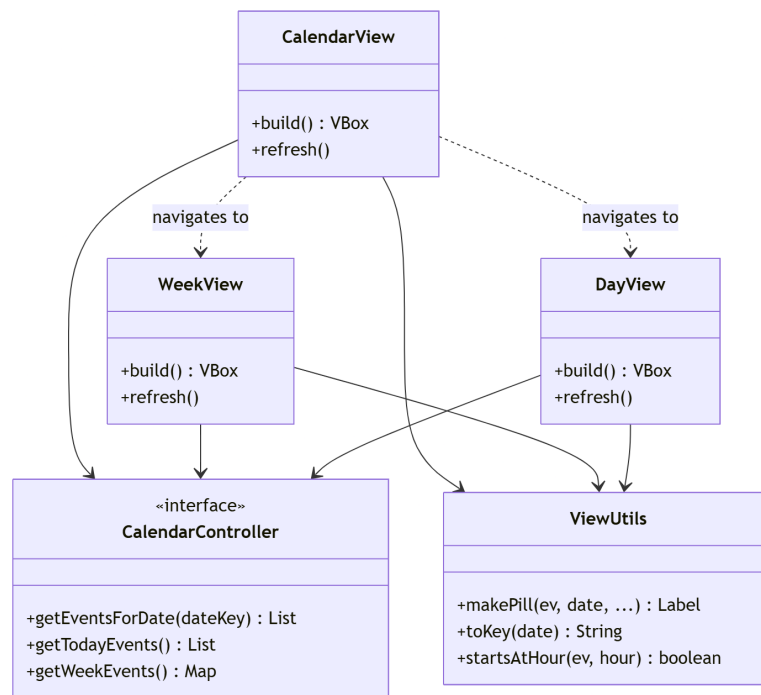


Figura 2.3: Le tre viste del calendario condividono il controller e sono navigate tramite toggle.

Bacheca con struttura gerarchica e associazione tag alle card

Problema La bacheca richiede una struttura a tre livelli (`Board` → `Column` → `Card`) con ordine di inserimento preservato anche dopo serializzazione e deserializzazione JSON. Le card devono poter avere tag associati per riferimento tramite id, essere marcate come completate, e filtrare la vista per tag attivi.

Soluzione L'ordine di inserimento è garantito tramite `LinkedHashMap` sia per le colonne che per le card, in `Board` e `Column` rispettivamente. Gson serializza le `LinkedHashMap` come oggetti JSON con chiavi stringa, preservando l'ordine. Il model `Card` include una lista di id di `KanbanTag` associati;

il filtraggio per tag attivi avviene nel controller tramite `getFilteredCards`, senza modificare il modello. L'intera struttura è contenuta in `BoardData`, un record immutabile analogo a `CalendarData`, che viene sostituito a ogni modifica e serializzato su disco tramite `BoardPersistence`. A differenza del calendario, non è stata introdotta un'interfaccia `BoardModel` separata in quanto le operazioni della bacheca sono prevalentemente CRUD su strutture dati ordinate, già testabili direttamente tramite `BoardControllerImpl` senza necessità di disaccoppiamento aggiuntivo. La struttura del model è mostrata in Figure 2.4.

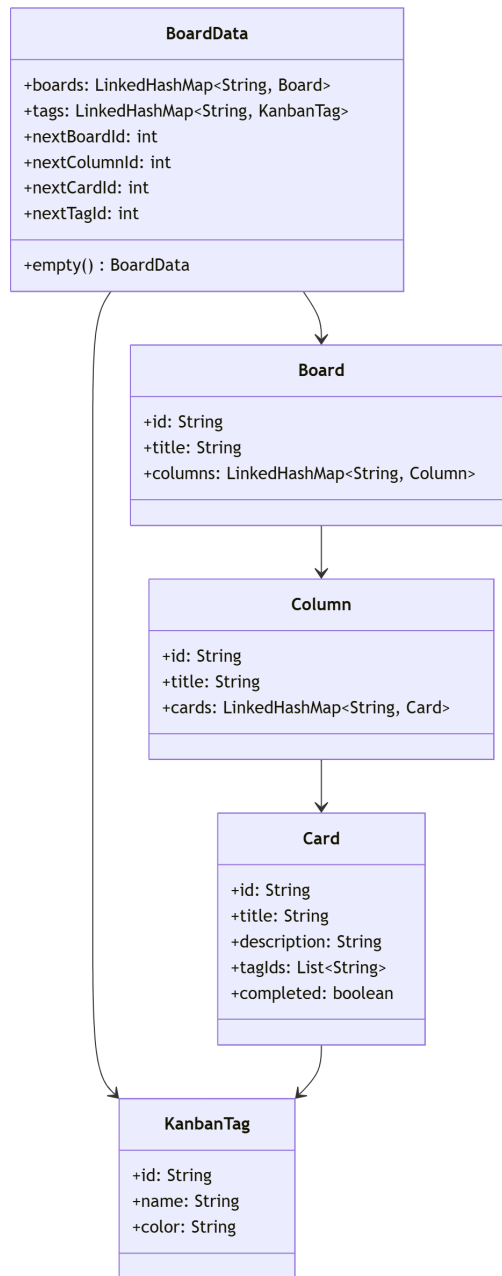


Figura 2.4: **BoardData** è un record immutabile che aggrega bacheche, colonne, card e tag.

2.2.2 Enrico Poli

Spostamento delle card tramite drag-and-drop

Problema Le card di una bacheca devono poter essere spostate dall'utente sia tra colonne diverse sia all'interno della stessa colonna, scegliendo la posizione di inserimento. Lo spostamento deve riflettersi nel modello e venire persistito, mantenendo l'ordine corretto delle card anche dopo un riavvio dell'applicazione.

Soluzione Lo spostamento è realizzato dal metodo `moveCard` di `BoardControllerImpl`, che riceve la bacheca, la colonna di origine, la colonna di destinazione, l'id della card e l'indice di inserimento. Poiché il modello è immutabile e le card sono memorizzate in `LinkedHashMap`, l'inserimento in una posizione arbitraria non è diretto: il metodo privato `insertAt` ricostruisce la mappa di destinazione nell'ordine voluto, scorrendo gli id esistenti e inserendo la card spostata all'indice indicato. Lo stesso meccanismo gestisce in modo unificato sia lo spostamento tra colonne diverse (la card viene rimossa dall'origine e inserita nella destinazione) sia il riordino interno alla stessa colonna (la card viene prima rimossa dalla sua vecchia posizione e poi reinserita). Se l'indice è fuori range la card viene posta in fondo alla colonna. Al termine la board aggiornata viene salvata tramite la persistenza, così che lo spostamento sopravviva al riavvio.

Le classi coinvolte sono mostrate in Figure 2.5.

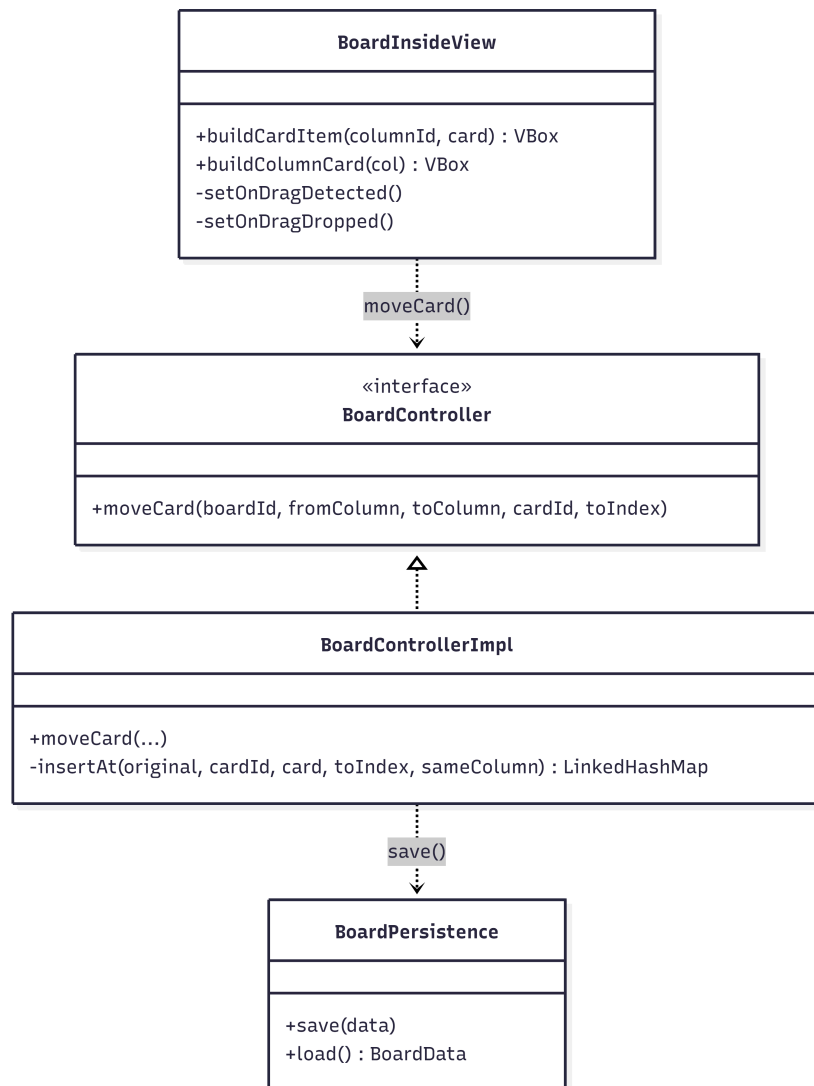


Figura 2.5: La view avvia il drag-and-drop e delega lo spostamento al controller, che ricostruisce l'ordine e persiste i dati..

Modulo budget: gestione e filtro dei tag

Problema Il modulo budget deve permettere di associare un tag categoria alle transazioni, di gestire i tag (creazione, modifica, eliminazione con scelta del colore) e di filtrare le transazioni mostrando solo quelle dei tag selezionati, in modo coerente con le sidebar di calendario e bacheche.

Soluzione La gestione dei tag del budget è centralizzata in una sidebar laterale (`TagSidebarView`) analoga a quelle degli altri moduli. Ogni tag è mostrato con una checkbox di filtro, un indicatore di colore e i comandi di modifica ed eliminazione; la creazione e la modifica avvengono tramite un dialog con `ColorPicker` per la personalizzazione del colore. Lo stato del filtro è mantenuto nel `BudgetController` come insieme di id di tag attivi, condiviso con la sidebar: i metodi `getIncomes` e `getExpenses` applicano il filtro, mostrando tutte le transazioni quando nessun tag è selezionato. Lo stesso pannello ospita il selettore del periodo, che filtra in modo unificato liste, totali e grafici.

Il collegamento è mostrato in Figure 2.6.

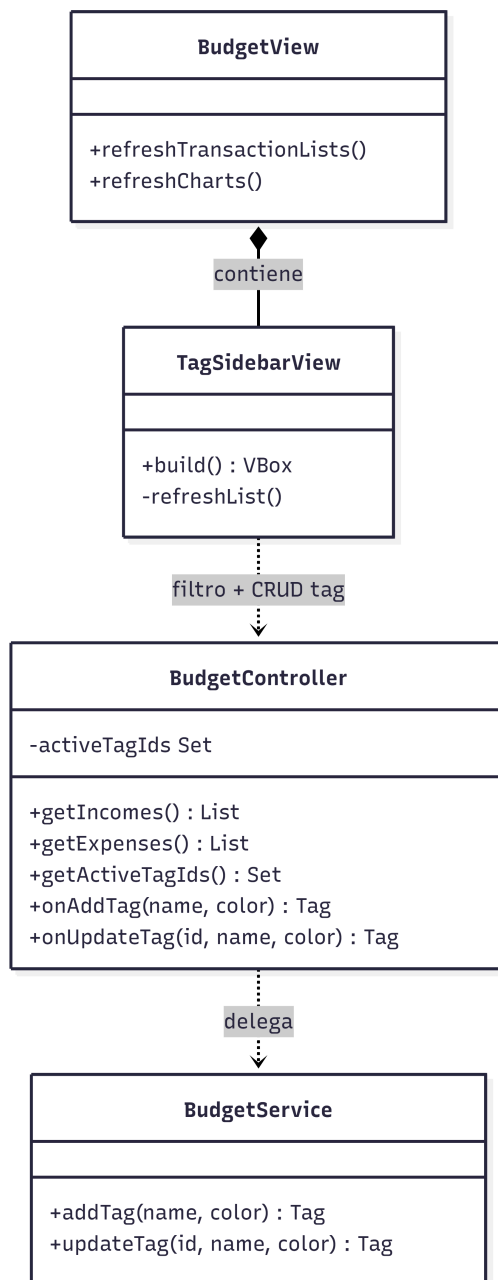


Figura 2.6: La sidebar dei tag del budget comunica col controller per il filtro e la gestione dei tag, che a sua volta delega la logica al service.

Calcolo dei totali e aggregazioni per i grafici

Problema Il modulo budget deve riepilogare le transazioni di un periodo mostrando entrate totali, uscite totali e saldo, e deve alimentare due grafici: la distribuzione delle uscite per categoria e l'andamento del saldo netto mese per mese. La logica di calcolo non deve risiedere nella view, che si limita a presentare i dati.

Soluzione Il calcolo è interamente delegato al `BudgetService`. Il metodo `calculateSummary` produce un `BudgetSummary`, un record immutabile che raccoglie entrate totali, uscite totali, saldo e le transazioni del periodo. L'aggregazione per i grafici è affidata a due metodi distinti: `aggregateByTag` somma le uscite del periodo raggruppandole per tag, fornendo i dati del grafico a torta; `aggregateByMonth` calcola il saldo netto di ogni mese, fornendo i dati del grafico a linee. Il `BudgetController` espone questi risultati alla view tramite metodi di sola lettura, così che `BudgetView` si limiti a popolare le etichette dei totali e i due grafici senza contenere alcuna logica di calcolo. Il saldo viene inoltre evidenziato con colori diversi a seconda che sia positivo, negativo o nullo.

Il flusso è mostrato in Figure 2.7.

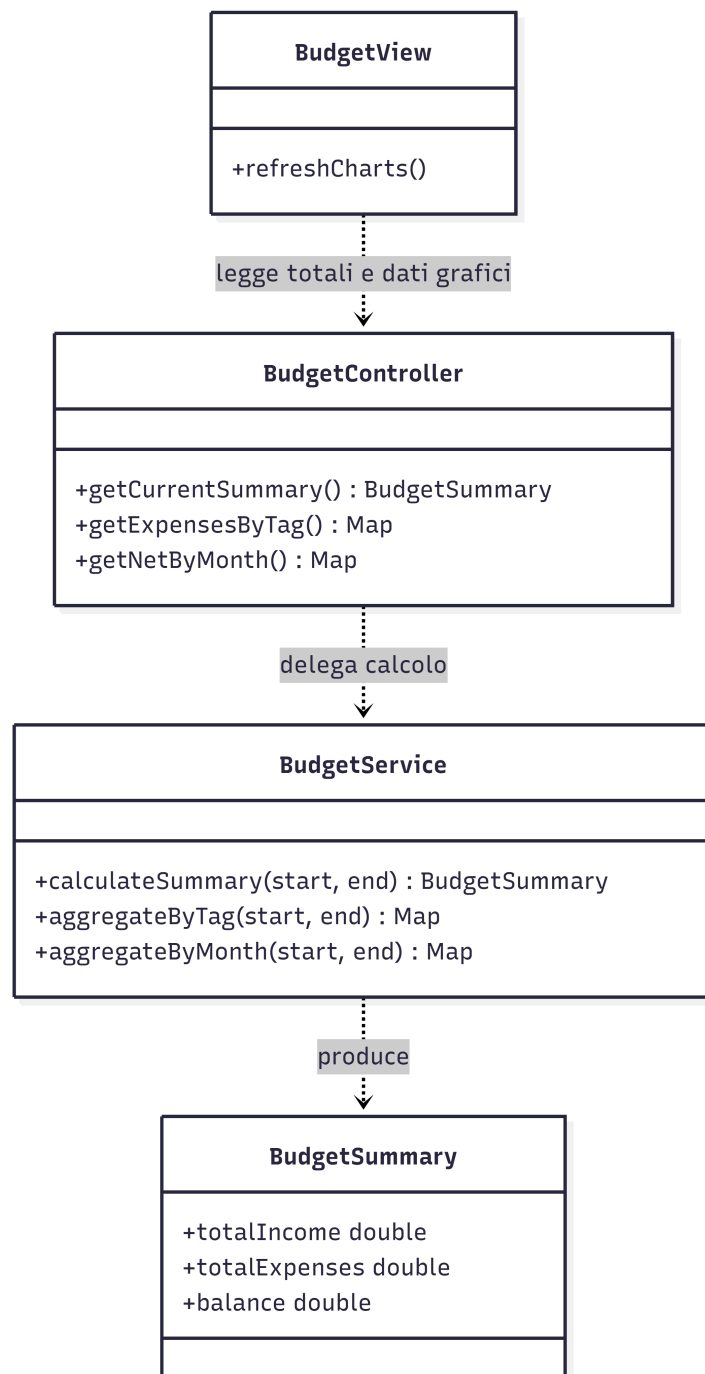


Figura 2.7: Il calcolo dei totali e l'aggregazione dei dati per i grafici sono delegati al service; la view si limita a leggerli tramite il controller.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Sono stati realizzati test automatizzati con JUnit per le componenti del model e del controller, in quanto privi di dipendenze grafiche e quindi testabili in isolamento. I test di calendario e kanban sono a cura di Aurora Loprino, mentre i test di budget e card move sono a cura di Enrico Poli.

Calendario (`com.chronio.calendar`)

- `DateValidationTest` verifica che la creazione di un evento con data di inizio nulla lanci `IllegalArgumentException`, che una data di fine precedente all'inizio lanci eccezione, e che date valide vengano accettate senza errori.
- `TagFilterTest` verifica che gli eventi con tag visibile appaiano nella vista per data, che quelli con tag nascosto vengano filtrati, e che gli eventi senza tag siano sempre visibili.
- `PersistenceTest` verifica che eventi e tag vengano correttamente serializzati su file JSON e ricaricati con i dati intatti, e che un file mancante restituisca uno stato vuoto.

Kanban (`com.chronio.kanban`)

- `BoardPersistenceTest` verifica che bacheche e colonne vengano ripristinate correttamente dopo un ciclo di salvataggio e ricaricamento.
- `ColumnDeletionTest` verifica che l'eliminazione di una colonna rimuova anche tutte le card contenute.

- **TagFilterTest** verifica che il filtro per tag restituisca solo le card associate al tag specificato, escludendo quelle prive di tag.
- **CardMoveTest** verifica che una card spostata in un'altra colonna non sia più presente nella colonna di origine e si trovi nella destinazione, e che lo spostamento sia persistito: dopo il ricaricamento dei dati da file la card risulta ancora nella colonna corretta, con i propri dati preservati.

Budget (`com.chronio.budget`)

- **BudgetSummaryTest** verifica che il saldo di un periodo corrisponda alla differenza tra entrate e uscite e che il filtro per periodo escluda correttamente le transazioni con data fuori dall'intervallo, includendo solo quelle comprese tra le date estreme.
- **ChartUpdateTest** verifica che i dati di alimentazione dei grafici si aggiornino dopo l'aggiunta di transazioni: l'aggregazione delle uscite per tag e il saldo netto mensile riflettono correttamente le nuove transazioni, inclusa la combinazione di entrate e uscite nello stesso mese.

3.2 Note di sviluppo

3.2.1 Aurora Loprino

Record Java come entità di dominio immutabili

Event, Tag, Board, Column, Card, KanbanTag sono tutti record Java.
[Permalink \(link diretto\)](#)

Lambda expressions

Usate nel model per filtraggio e trasformazione degli eventi. Esempio in `data.events().forEach((id, ev)`.
[Permalink \(link diretto\)](#)

Optional

Usato nelle operazioni di aggiornamento del model (`updateEvent`, `updateTag`, `updateCard`) per segnalare l'assenza dell'entità senza eccezioni.
[Permalink \(link diretto\)](#)

Libreria Gson (Google)

Usata per la serializzazione/deserializzazione JSON dei dati di calendario e kanban in `CalendarPersistence` e `BoardPersistence`.

[Permalink \(link diretto\)](#)

Stream

Utilizzati in `CalendarView` per limitare e iterare gli eventi visibili per cella nella vista mensile.

[Permalink \(link diretto\)](#)

Libreria JavaFX

Usata per l'intera interfaccia grafica. Esempio in `ViewUtils.makePill` che utilizza `Tooltip` con `setShowDelay(Duration.ZERO)` per mostrare la descrizione dell'evento all'hover del mouse immediatamente (solo se la descrizione è presente.)

[Permalink \(link diretto\)](#)

3.2.2 Enrico Poli

Drag-and-drop di JavaFX

Utilizzato in `BoardInsideView` per lo spostamento delle card tramite `setOnDragDetected`, `setOnDragOver` e `setOnDragDropped`, con trasporto dei dati tramite *drag-board*.

[Permalink \(link diretto\)](#)

LinkedHashMap per l'ordinamento

Sfruttata in `moveCard` e `insertAt` per ricostruire l'ordine delle card preservando le posizioni durante lo spostamento e il riordino.

[Permalink \(link diretto\)](#)

Stream

Utilizzati nel `BudgetController` per filtrare le transazioni per tipo (entrata/uscita) e per tag attivo nei metodi `getIncomes` e `getExpenses`.

[Permalink \(link diretto\)](#)

Libreria JavaFX

Usata per la sidebar dei tag del budget (`TagSidebarView`), con `ColorPicker` per la scelta del colore e `Tooltip` per mostrare il nome del tag all'hover.

[Permalink](#) (link diretto)

Libreria Gson (Google)

Sfruttata indirettamente per la persistenza JSON degli spostamenti di card, con adapter per la serializzazione del tipo `LocalDate` nelle transazioni del budget.

[Permalink](#) (link diretto)

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

Aurora Loprino

Chronio è nato come progetto per il corso di Ingegneria Web e abbiamo colto l'occasione per svilupparlo ulteriormente. Mi piacerebbe renderlo utilizzabile in altri linguaggi, fino alla tesi di laurea. Non ho mai programmato in Java ed apprendere il linguaggio è stato impegnativo ma molto soddisfacente. In futuro mi piacerebbe lavorare nell'ambito di DevOps, quindi spero di mettere in pratica ed implementare sempre di più quanto appreso.

Inizialmente il lavoro era ripartito tra tre persone. Ho avuto delle difficoltà oggettive nella gestione improvvisa dell'aumento del carico di lavoro, quando a due settimane dalla consegna ci siamo fatti carico della parte scoperta. Ho utilizzato Chronio per calendarizzare le attività e tenere traccia di quanto fatto e quanto mancava, sperimentandone i benefici in prima persona.

Purtroppo per mancanza di tempo non abbiamo potuto assegnare a Chronio la sua palette di colori, ma nel futuro si potrebbe pensare di abbellirlo e magari dargli un backend condiviso con l'applicazione web.

Enrico Poli

Sono ormai affezionato al progetto Chronio, dato che ero in gruppo con Aurora anche per Ingegneria dei Sistemi Web, e sono contento che abbiamo deciso di svilupparlo anche in Java. Questa è stata la prima volta che ho usato questo linguaggio e, nonostante sia difficile iniziare con un progetto secondo me già abbastanza avanzato, mi è sembrato più intuitivo di altri linguaggi. Mi piacerebbe continuare a usarlo in futuro, anche solo per progetti personali

come semplici applicazioni desktop, per analisi sui miei dati personali, o per integrarlo con Raspberry nell'ambito della domotica.

Ci sono state alcune difficoltà oggettive con il carico di lavoro quando, a due settimane dalla consegna, il gruppo è passato da tre a due persone, lasciando scoperta una parte considerevole. Ho avuto qualche incertezza sulla possibilità di completare il progetto entro la scadenza, sia per motivi di tempo sia per la difficoltà di comprendere alcuni bug.

Ho usato Chronio anche in ambito personale, principalmente per tenere sotto controllo i miei movimenti finanziari. In futuro mi piacerebbe rendere il modulo budget più completo, magari inserendo grafici più articolati o un calcolo del budget basato su parametri più vari, come la situazione familiare o patrimoniale.

Appendice A

Guida utente

All'avvio, Chronio mostra tre sezioni principali accessibili dalla barra di navigazione superiore: **Calendario**, **Bacheche** e **Budget**.

Calendario

- Clic su una cella del mese per aggiungere un nuovo evento.
- Clic su una pill per modificare o eliminare un evento esistente.
- I pulsanti < e > navigano tra i mesi.
- Il pulsante “Settimana” / “Giorno” / “Mese” alterna la vista.
- Nella sidebar a sinistra, il pannello “Tags” permette di creare, modificare, eliminare e mostrare/nascondere i tag. I tag nascosti filtrano gli eventi dalla vista.
- I pannelli “Oggi” e “Questa settimana” mostrano gli eventi imminenti.
- Tramite hover si può leggere la descrizione dell'evento.

Bacheche

- Clic su “+ Nuova Bacheca” per creare una bacheca.
- All'interno di una bacheca, è possibile aggiungere colonne e card tramite i pulsanti dedicati.
- Le card possono essere marcate come completate tramite checkbox.

- La sidebar dei tag permette di filtrare le card per tag.
- Tramite hover si può leggere la descrizione della card o il nome del tag assegnato.
- Le card possono essere spostate trascinandole con il mouse, sia tra colonne diverse sia all'interno della stessa colonna.
- Rilasciando una card sulla metà superiore di un'altra, viene inserita prima di essa; sulla metà inferiore, viene inserita dopo. Rilasciandola nello spazio vuoto di una colonna, viene spostata in fondo.

Budget

- I movimenti sono divisi in due colonne: Entrate e Uscite.
- Clic su “+ Aggiungi transazione” in fondo a una colonna per inserire un nuovo movimento, indicando descrizione, importo, data e tag.
- Clic sull'icona di modifica di una card per modificarne i dati, o sull'icona di eliminazione per rimuoverla.
- Nella sidebar a sinistra, il pannello “Tags” permette di creare, modificare ed eliminare i tag, scegliendone il colore. Le checkbox filtrano i movimenti mostrando solo quelli dei tag selezionati.
- Sempre nella sidebar, i selettori di periodo “Da” e “A” filtrano i movimenti, i totali e i grafici, mostrando solo le transazioni comprese nell'intervallo scelto.
- Il pannello “Totale” mostra entrate totali, uscite totali e saldo del periodo, con il saldo evidenziato in verde se positivo e in rosso se negativo.
- Il grafico a torta riepiloga le uscite per categoria, mentre il grafico a linee mostra l'andamento del saldo netto mese per mese.
- Tramite hover sul pallino colorato di una transazione si può leggere il nome del tag assegnato.